COLLÈGE AHUNTSIC

Département d'informatique

420-315-AH — Développement d'objets intelligents

Projet 2 — Évaluation intégratrice

Assistant vocal de lampe intelligente

1. Contexte

Une entreprise en domotique souhaite valider rapidement un **prototype d'assistant vocal local** pour une chambre intelligente. Le client ne cherche pas un produit commercial complet, mais un démonstrateur fiable, clair et bien structuré, capable de prouver qu'une interaction vocale simple peut piloter un objet connecté sur Raspberry Pi.

Le besoin d'affaires est le suivant : permettre à une personne d'interagir naturellement avec une **lampe intelligente** à la voix, sans interface graphique complexe, en utilisant un mot d'activation, une reconnaissance de commande, une communication MQTT et une action réelle sur le montage. Le système doit également fournir un retour vocal à l'utilisateur et conserver une trace minimale des actions exécutées.

Ce projet s'inscrit directement dans la partie du cours consacrée à l'**assistant vocal IoT**, qui vise l'intégration de la synthèse vocale, de la reconnaissance vocale, de l'interprétation de commandes, de MQTT et d'une persistance minimale. Le Projet 2 est prévu comme évaluation intégratrice, réalisé **en équipe de 2**, avec **démonstration fonctionnelle** et **courte défense individuelle**, pour une remise et des démonstrations à la **période 12, le 10 avril 2026**. Il vaut **40 %** de la note finale du cours.

2. Mandat

Vous agissez comme une équipe de développement chargée de livrer un prototype fonctionnel nommé :

Assistant vocal de lampe intelligente

Votre solution doit permettre à un utilisateur de contrôler une lampe représentée par une DEL ou un montage équivalent sur Raspberry Pi à l'aide de commandes vocales simples.

3. Objectif général

Concevoir et démontrer un assistant vocal capable de :

- reconnaître un mot d'activation ;
- écouter une commande vocale ;
- convertir l'audio en texte ;

- interpréter la commande ;
- publier l'action via MQTT ;
- exécuter l'action sur la lampe ;
- fournir un retour vocal ;
- journaliser minimalement les commandes et résultats.

4. Commandes obligatoires

Votre système doit obligatoirement prendre en charge les commandes suivantes :

- **allume la lampe**
- **éteins la lampe**
- **fais clignoter la lampe**
- **donne-moi l'état**
- **active le mode nuit**

Vous pouvez ajouter d'autres commandes, mais ces cinq commandes sont obligatoires.

5. Fonctionnement attendu

Le scénario minimal attendu est le suivant :

1. l'utilisateur prononce le mot d'activation ;
2. le système confirme qu'il est en écoute ;
3. l'utilisateur prononce une commande ;
4. la commande est transcrite en texte ;
5. l'intention est déterminée ;
6. l'action est publiée sur MQTT ;
7. un composant ou subscriber exécute l'action sur la lampe ;
8. le système retourne un message vocal ou texte de confirmation ;
9. une trace minimale de la commande est conservée.

Exemple de scénario :

- Utilisateur : **assistant**
- Système : **Je vous écoute**
- Utilisateur : **allume la lampe**
- Système : **Lampe allumée**

6. Exigences obligatoires

6.1 Mot d'activation

Vous devez intégrer un **hot-word** simple, par exemple assistant, système ou raspberry. Le mot d'activation doit déclencher l'écoute de la commande.

6.2 Reconnaissance vocale

Le système doit :

- écouter une phrase prononcée ;
- convertir cette phrase en texte ;
- afficher le texte reconnu ;
- gérer au minimum :
  - le silence ;
  - le bruit ;
  - les timeouts.

6.3 Synthèse vocale

Le système doit inclure une fonction de type : speak(text)

Le retour vocal doit être utilisé pour :

- confirmer une action ;
- signaler une erreur ;
- confirmer l'écoute.

6.4 Interprétation des commandes

Le texte reconnu doit être transformé en **intention**.

Exigences minimales :

- au moins **3 expressions régulières robustes** ;
- une logique claire de transformation **texte → intention** ;
- une séparation lisible entre la partie reconnaissance et la partie interprétation.

6.5 Communication MQTT

Le système doit démontrer un passage réel par MQTT entre :

- l'assistant vocal ;
- le module qui exécute l'action sur la lampe.

6.6 Action réelle sur la lampe

Le projet doit démontrer une action réelle sur Raspberry Pi.

La lampe intelligente doit au minimum permettre :

- l'allumage ;
- l'extinction ;

- le clignotement ;
- le mode nuit.

Le mode nuit peut être simple, par exemple :

- clignotement lent ;
- intensité réduite ;
- comportement clairement défini et démontré.

6.7 Journalisation minimale

Vous devez enregistrer au minimum :

- la commande texte ;
- l'intention détectée ;
- la date et l'heure ;
- le résultat ou l'état.

6.8 Documentation

Vous devez fournir un **README** clair contenant :

- la description du projet ;
- le matériel utilisé ;
- les dépendances ;
- les étapes d'installation ;
- la procédure de lancement ;
- les commandes supportées ;
- la structure du projet.

Le plan de cours demande explicitement de documenter l'installation, les dépendances, de préparer une démo « scénario utilisateur » et de stabiliser le pipeline complet.

7. Livrables

7.1 Code source complet

Le projet remis doit contenir au minimum :

- le module TTS ;
- le module STT ;
- le module hot-word ;
- le module d'interprétation ;
- le module MQTT ;
- le module d'action sur GPIO ;
- le module de journalisation ;
- le README.

7.2 Démonstration fonctionnelle

Vous devez présenter une démonstration fonctionnelle en classe.

La démonstration doit montrer au minimum :

- le mot d'activation ;
- plusieurs commandes obligatoires ;
- la communication MQTT ;
- l'action réelle sur la lampe ;
- le retour vocal ;
- la journalisation minimale.

7.3 Défense individuelle

Chaque membre de l'équipe doit être capable d'expliquer :

- sa contribution ;
- la structure du système ;
- les choix techniques ;
- les difficultés rencontrées ;
- les limites du projet.

8. Organisation du travail

Le projet est réalisé **en équipe de 2**. La contribution individuelle doit être identifiable. La remise et la démonstration du Projet 2 sont prévues le **10 avril 2026**, avec **démo fonctionnelle par équipe** et **courte défense individuelle**.

9. Dates importantes

Présentation du projet : 10 avril 2026
Remise du code : 10 avril 2026, en fin de journée

10. Politique de retard

Les travaux doivent être remis à l'échéance fixée. Pour ce projet, une pénalité de **10 % par tranche de 24 heures de retard** s'applique à la note obtenue. Aucun travail n'est accepté après la diffusion du solutionnaire, des copies corrigées ou des résultats. Cette règle est cohérente avec la note générale du plan de cours.

Exemples :

- 1 jour de retard : **-10 %**
- 2 jours de retard : **-20 %**
- 3 jours de retard : **-30 %**

11. Barème d'évaluation – sur 100 points

Critère d'évaluation	Points précis à évaluer	Échelle d'appréciation	Pondération
1. Synthèse vocale (TTS)	Présence d'une fonction speak(text) ; confirmations vocales ; messages d'erreur ; clarté et pertinence du feedback ; intégration dans le scénario utilisateur	**Excellent : 9 à 10 pts** — TTS complet, clair, utile et bien intégré. **Bien : 7 à 8 pts** — TTS fonctionnel avec quelques limites mineures. **Passable : 5 à 6 pts** — TTS partiel mais exploitable. **Insuffisant : 0 à 4 pts** — TTS absent ou non fonctionnel.	**10**
2. Reconnaissance vocale (STT)	Capture audio fonctionnelle ; choix correct du micro ; réglages adéquats ; transcription cohérente ; stabilité générale	**Excellent : 13 à 15 pts** — STT stable, précis, bien réglé. **Bien : 10 à 12 pts** — STT fonctionnel avec quelques ratés. **Passable : 7 à 9 pts** — STT irrégulier mais partiellement utilisable. **Insuffisant : 0 à 6 pts** — STT absent ou très faible.	**15**
3. Hot-word	Présence d'un mot d'activation ; détection fonctionnelle ; déclenchement correct de l'écoute ; gestion minimale des échecs	**Excellent : 9 à 10 pts** — Hot-word fiable et bien intégré. **Bien : 7 à 8 pts** — Fonctionnel avec limites mineures. **Passable : 5 à 6 pts** — Partiel ou instable. **Insuffisant : 0 à 4 pts** — Absent ou non fonctionnel.	**10**
4. Interprétation des commandes	Reconnaissance des 5 commandes obligatoires ; présence d'au moins 3 regex robustes ; mapping texte → intention ; cohérence de l'analyse	**Excellent : 17 à 20 pts** — Interprétation claire, robuste, bien structurée. **Bien : 13 à 16 pts** — Fonctionnelle avec quelques cas limites. **Passable : 9 à 12 pts** — Partielle ou fragile. **Insuffisant : 0 à 8 pts** — Faible ou absente.	**20**

5. Intégration MQTT	Publication correcte ; circulation claire des messages ; communication réelle entre les modules ; structure logique des topics	**Excellent : 13 à 15 pts** — MQTT stable, clair, bien structuré. **Bien : 10 à 12 pts** — Fonctionnel avec petites faiblesses. **Passable : 7 à 9 pts** — Partiel ou fragile. **Insuffisant : 0 à 6 pts** — Absent ou non fonctionnel.	**15**
6. Action sur la lampe intelligente	allume la lampe ; éteins la lampe ; fais clignoter la lampe ; active le mode nuit ; cohérence entre commande, message MQTT et action physique	**Excellent : 13 à 15 pts** — Toutes les actions sont fonctionnelles, stables et bien démontrées. **Bien : 10 à 12 pts** — Actions globalement fonctionnelles. **Passable : 7 à 9 pts** — Certaines actions seulement fonctionnent correctement. **Insuffisant : 0 à 6 pts** — Actions absentes ou incomplètes.	**15**
7. Commande "donne-moi l'état"	Reconnaissance de la commande ; retour cohérent sur l'état actuel ; réponse claire à l'écran et/ou à l'oral	**Excellent : 5 pts** — État exact, clair, bien formulé. **Bien : 4 pts** — État correct avec légère imprécision. **Passable : 2 à 3 pts** — État partiel ou peu clair. **Insuffisant : 0 à 1 pt** — État absent ou faux.	**5**
8. Journalisation minimale	Commande enregistrée ; intention enregistrée ; horodatage ; résultat ou état enregistré	**Excellent : 5 pts** — Journalisation complète, claire, exploitable. **Bien : 4 pts** — Présente mais légèrement incomplète. **Passable : 2 à 3 pts** — Partielle. **Insuffisant : 0 à 1 pt** — Absente ou inutilisable.	**5**
9. Qualité technique du code	Structure des fichiers ; lisibilité ; commentaires ; gestion des exceptions ; séparation des responsabilités ; robustesse minimale	**Excellent : 9 à 10 pts** — Code propre, structuré, robuste. **Bien : 7 à 8 pts** — Code correct avec quelques faiblesses mineures.	**10**

		Passable : 5 à 6 pts — Peu structuré mais compréhensible. **Insuffisant : 0 à 4 pts** — Confus ou instable.	
10. Documentation, présentation et défense	README complet ; installation reproductible ; démonstration structurée ; scénario utilisateur clair ; qualité des réponses en défense	**Excellent : 13 à 15 pts** — Documentation complète, démo maîtrisée, défense solide. **Bien : 10 à 12 pts** — Ensemble clair avec imprécisions mineures. **Passable : 7 à 9 pts** — Documentation ou explications incomplètes. **Insuffisant : 0 à 6 pts** — Faible ou confus.	**15**

Total : 100 points

12. Rappel institutionnel

Le Projet 2 est une évaluation intégratrice de fin de cours. Il vise à intégrer les apprentissages liés à **MQTT**, **Raspberry Pi**, **TTS/STT**, **parsing regex/NLTK** et **persistance minimale**, afin de livrer un assistant vocal capable d'exécuter des commandes sur un objet via MQTT, avec feedback vocal et journalisation.

13. Recommandation de réalisation

Il est fortement recommandé de développer le projet par étapes :

1. TTS
2. STT
3. hot-word
4. interprétation
5. MQTT
6. action sur la lampe
7. journalisation
8. démonstration complète

14. Mention finale

Le succès de ce projet repose sur une approche progressive, des tests fréquents, une bonne séparation des modules et une démonstration simple, stable et convaincante.